Patroiak Proiektua 26

Factory Method, Iterator y Adapter. YerayOrdoki/proiektua2, rama patroiak-proiektua26.

Requisitos: patroiakProiektua26.pdf, paginas 1-5. Base revisada: 05510415231a40dfa3946b4ce976957e9edc9503.

Verificacion automatica: {'tests': 9, 'failures': 0, 'errors': 0, 'skipped': 0}. Capturas de ejecucion Swing con fixtures deterministas, no datos personales ni pruebas SOAP extremo a extremo.

Repositorio: YerayOrdoki/proiektua2. Rama: patroiak-proiektua26. Base: 05510415231a40dfa3946b4ce976957e9edc9503. Requisitos: patroiakProiektua26.pdf, paginas 1-5.

Factory Method Creator: BLFactory.Creator. Factory Method: create(). ConcreteCreator: LocalCreator y RemoteCreator. Product: BLFacade. ConcreteProduct: BLFacadeImplementation y RemoteFacade. BLFactory selecciona el Creator segun el booleano recibido de la presentacion. No es solo un if: cada Creator sobrescribe create. Se pueden inyectar creadores que devuelvan mocks de Mockito. ApplicationLauncher lee ConfigXML y obtiene la fachada mediante la factoria, sin construir DataAccess ni configurar SOAP. La red y la DB se resuelven fuera del EDT; las ventanas se crean dentro del EDT.

Iterator ExtendedIterator<T> extiende Iterator<T>. ListExtendedIterator copia la pertenencia a la lista, no clona los Sale. El cursor indica el elemento actual: next devuelve ese elemento e incrementa; previous lo devuelve y decrementa. goFirst establece 0; goLast establece size-1 para no omitir el ultimo elemento. No es la semantica de ListIterator. Mezclar next y previous sin reposicionar sigue ese mismo cursor, segun el comentario del PDF. El agotamiento lanza NoSuchElementException; remove no esta permitido. Se cubren listas vacias, un elemento, cambios de direccion y snapshot. BLFacade y BLFacadeImplementation exponen getSalesIterator(String desc) reutilizando getSales(desc). La sobrecarga sin parametros usa la busqueda vacia, por la inconsistencia entre firma y ejemplo del enunciado. PatternsDemo imprime primero el recorrido inverso y despues el directo.

SOAP y compatibilidad El iterador es estado del cliente y no se serializa por SOAP. RemoteBLFacade conserva las 33 operaciones originales y el nombre XML BLFacade. BLFacade extiende ese contrato y agrega la iteracion local. BLFacadeImplementation publica RemoteBLFacade. RemoteFacade delega las operaciones del servicio y hereda la construccion local del iterador. Es necesario reiniciar el servidor con el codigo de esta rama. Las pruebas con mocks no certifican SOAP extremo a extremo ni ObjectDB real. BLFacadeImplementation conserva sus operaciones anteriores, reformateadas, cambia endpointInterface y agrega getSalesIterator. No se corrigen otros problemas historicos de conexiones.

Adapter Target: TableModel. AbstractTableModel implementa parte de ese contrato. Adapter: SellerAdapter. Adaptee: Seller y sus Sale. Client: JTable, creada dentro de SellerTable. getRowCount corresponde al numero de ventas; getValueAt traduce los getters de Sale a celdas. Las columnas son Sale, Title, Description, Status, Price y Publication date; sus tipos son Integer, String, String, Integer, Float y Date, incluso sin filas. El modelo no permite editar y copia la pertenencia a la lista. Seller.sales es XmlTransient: el ejemplo remoto carga getSalesBySeller(email) y usa la sobrecarga que recibe la lista. isLogged devuelve Emaitza en este proyecto, no Seller; la demostracion resuelve el vendedor por email en lugar de copiar ese error del ejemplo del PDF. La extraccion textual del PDF no permite verificar las cabeceras de su imagen: se documentan expresamente las seis columnas elegidas.

Ejecutar la aplicacion Requisitos: JDK 11 y Maven. No se cambia config.xml ni se sobrescriben bases de datos. Para trabajar localmente, configurar businessLogic local="true". Para inicializar los datos del enunciado, usar initialize="true" en una copia de desarrollo, arrancar una vez y devolverlo a false. No se hace automaticamente porque puede afectar a datos existentes. En modo remoto, configurar host y puerto alcanzables, mantener local="false" y arrancar businessLogic.BusinessLogicServer antes del cliente. Los recursos y dependencias existentes del proyecto deben estar en el classpath de ejecucion.

mvn -B package -DskipTests mvn -q org.codehaus.mojo:exec-maven-plugin:3.1.0:java -Dexec.mainClass=patterns.demo.PatternsDemo -Dexec.args="seller1@gmail.com"

El argumento es el email de un vendedor existente; el segundo argumento opcional filtra las ventas del iterador. La ventana muestra todas las ventas del vendedor. Se puede ejecutar patterns.demo.PatternsDemo desde Eclipse con los mismos argumentos.

Pruebas y evidencias PatternsTest contiene ocho pruebas: ordenes y limites, vacio/un elemento, snapshot/cambio de direccion, seleccion de Creator con Mockito, busqueda local con DataAccess mock, iterador remoto cliente con mock SOAP, mapeo/tipos del Adapter y vendedores vacios/remotos. PatternsScreenshotTest agrega una prueba con datos deterministas no persistidos que ejecuta los ejemplos y captura ventanas Swing reales mediante Robot bajo Xvfb. No son imagenes inventadas ni capturas de los datos personales del usuario. Sin servidor grafico la prueba se omite y la entrega se rechaza como no validada.

xvfb-run -a mvn -B test -Dtest=PatternsTest,PatternsScreenshotTest plantuml -charset UTF-8 -tpng doc/patterns/*.puml pip install reportlab python-docx python tools/build-patterns-docs.py

El workflow patterns-delivery comprueba los XML de Surefire: al menos nueve pruebas y cero fallos, errores u omisiones. Es importante porque el pom historico ignora fallos de test. Solo con las pruebas y capturas presentes genera patters.pdf en la raiz y doc/patterns/patroiak.docx, con UML ampliado, explicaciones, capturas y codigo fuente numerado. Guarda esos resultados automaticamente en esta misma rama, nunca en master. Los informes

se conservan como artefactos de Actions incluso si falla la ejecucion. Un workflow preparado no equivale a una ejecucion exitosa: consultar validation.json y los informes generados.

Entrega y OneDrive El enunciado exige entregar la URL del proyecto y tener patters.pdf en el repositorio. El Word es adicional, compatible con Word de OneDrive. No se ha subido a OneDrive: la carga directa requiere conectar Microsoft 365. Los documentos de esta automatizacion solo se generan tras validar las pruebas; los borradores adjuntos anteriores no certifican ejecucion. Quedan fuera de la validacion automatica: SOAP extremo a extremo, ObjectDB real y coincidencia exacta de cabeceras con la imagen del enunciado.

UML ampliado

Factory Method

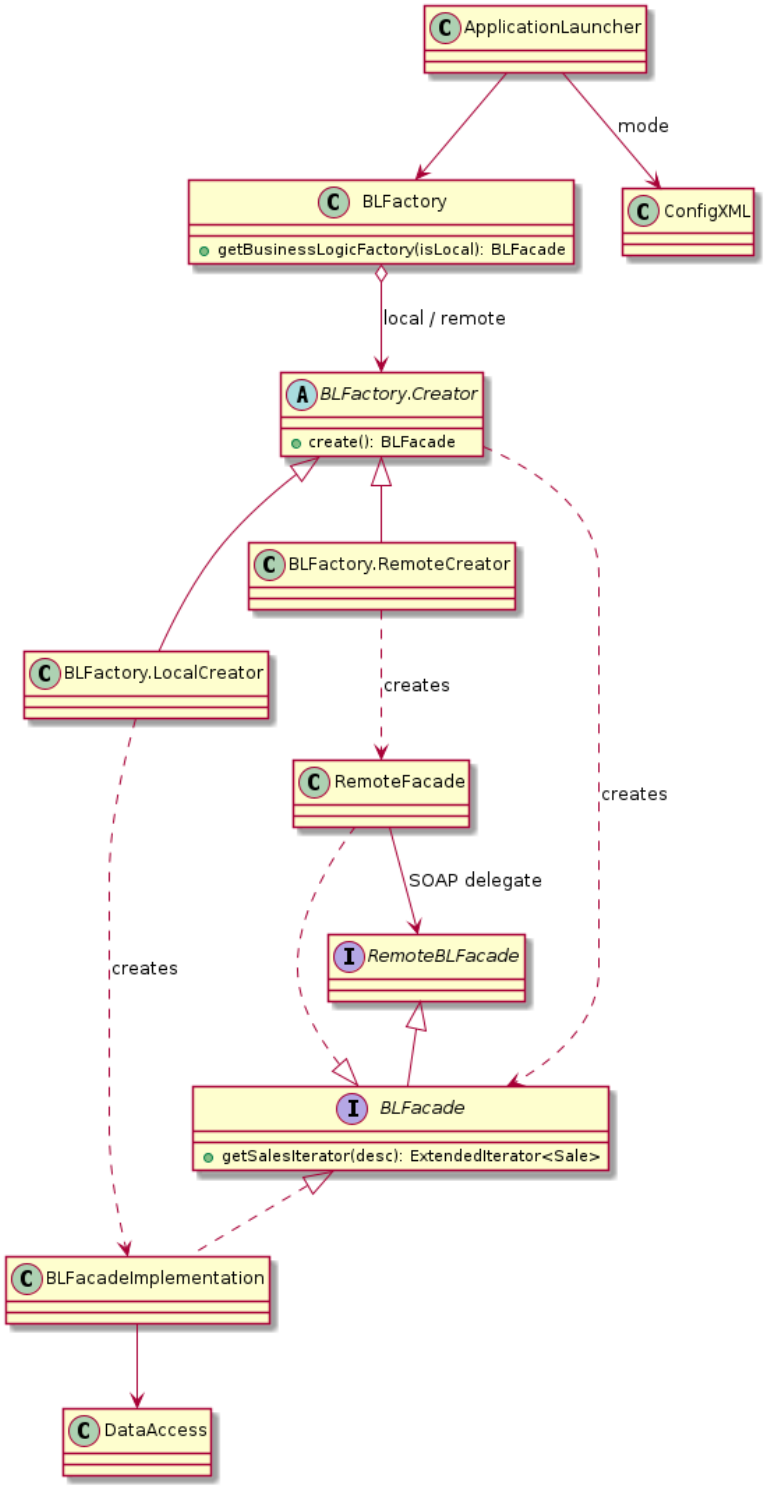


Diagrama UML con las clases y relaciones implementadas; fuente editable en el archivo .puml correspondiente.

Iterator

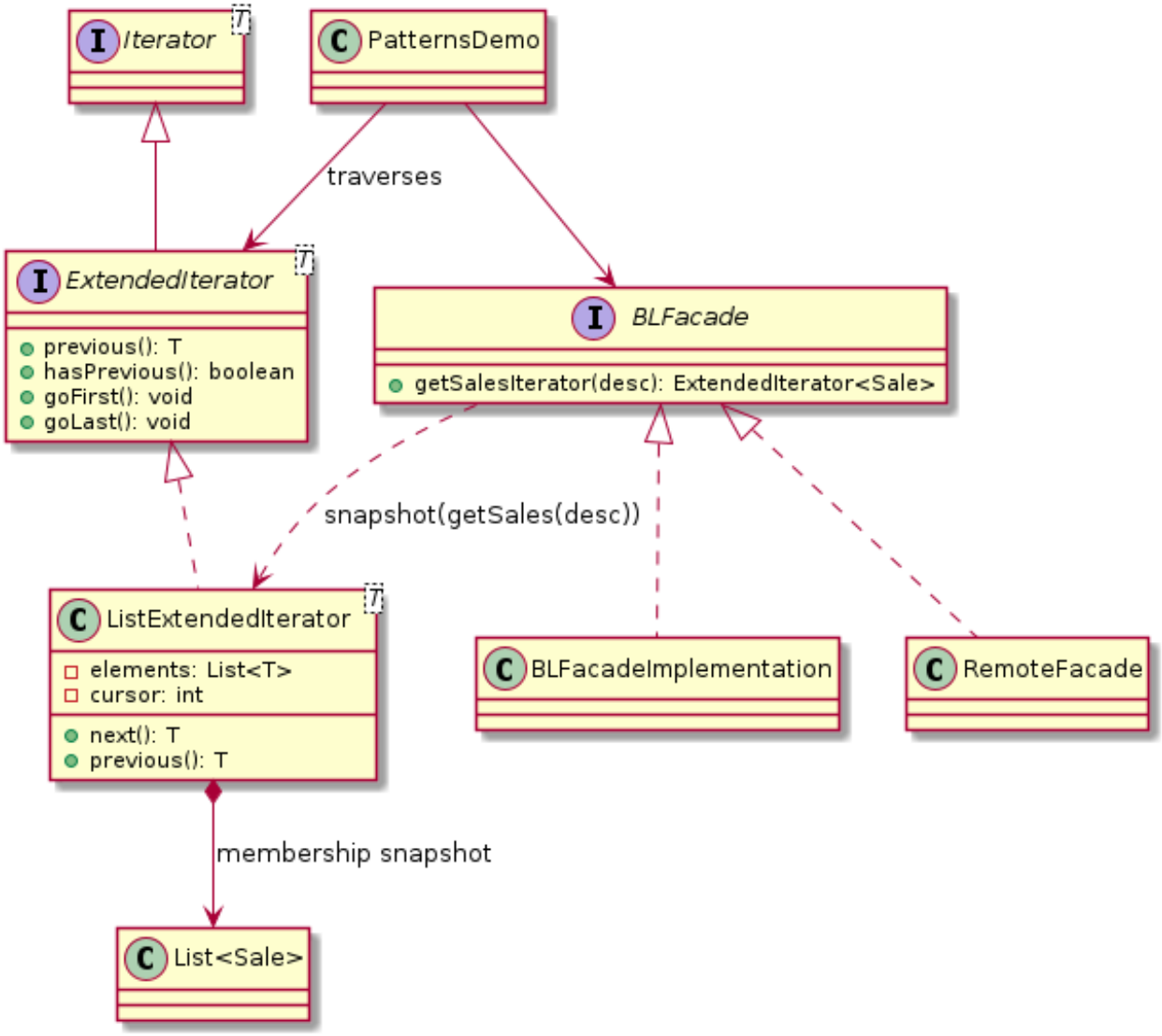


Diagrama UML con las clases y relaciones implementadas; fuente editable en el archivo .puml correspondiente.

Adapter

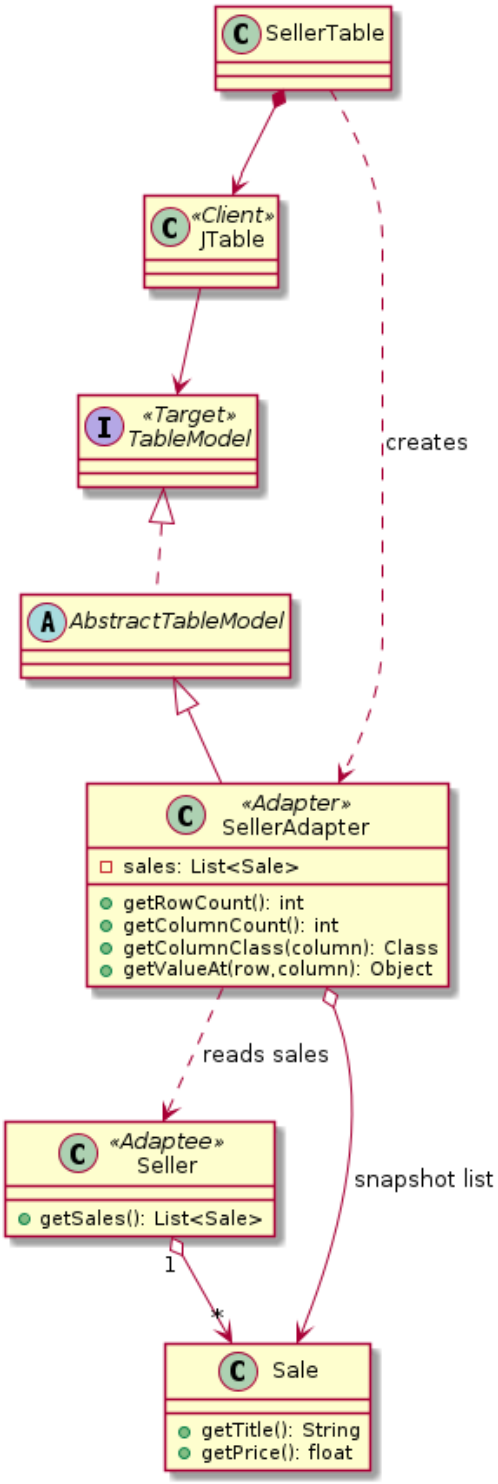


Diagrama UML con las clases y relaciones implementadas; fuente editable en el archivo .puml correspondiente.

Evidencias de ejecucion

```
FROM LAST TO FIRST
3;Keyboard;35.0
2;Monitor;85.0
1;Bicycle;120.0

FROM FIRST TO LAST
1;Bicycle;120.0
2;Monitor;85.0
3;Keyboard;35.0
```

Iterator: primero del ultimo al primero y despues del primero al ultimo, ejecutando PatternsDemo.describe.

Sale	Title	Description	Status	Price	Publication date
1	Bicycle	City bicycle	1	120	Jan 1, 1970
2	Monitor	24 inch	0	85	Jan 2, 1970
3	Keyboard	Mechanical	1	35	Jan 3, 1970

Adapter: ventana SellerTable y JTable reales usando SellerAdapter; datos de demostracion no persistidos.

Codigo entregado y lineas importantes

src/main/java/businessLogic/BLFacade.java

getSalesIterator(desc) reutiliza getSales(desc). La sobrecarga sin argumentos usa busqueda vacia.

```
 1 package businessLogic;
 2 import domain.Sale;
 3 import patterns.iterator.ExtendedIterator;
 4 import patterns.iterator.ListExtendedIterator;
 5 /** Product shared by local logic and the remote client facade. */
 6 public interface BLFacade extends RemoteBLFacade {
 7     /** The iterator is local state, never a SOAP return value. */
 8     default ExtendedIterator<Sale> getSalesIterator(String desc) {
 9         return new ListExtendedIterator<Sale>(getSales(desc));
10     }
11     default ExtendedIterator<Sale> getSalesIterator() {
12         return getSalesIterator("");
13     }
14 }
```

src/main/java/businessLogic/BLFacadeImplementation.java

Cambio del endpointInterface a RemoteBLFacade y adicion de getSalesIterator(String); operaciones anteriores conservadas, con reformateo.

```
 1 package businessLogic;
 2 import java.io.File;
 3 import java.util.Date;
 4 import java.util.List;
 5 import javax.jws.WebMethod;
 6 import javax.jws.WebService;
 7 import dataAccess.*;
 8 import dataAccess.DataAccess;
 9 import domain.Complaint;
10 import domain.Eskaera;
11 import domain.EskaeraOferta;
12 import domain.Review;
13 import domain.Sale;
14 import dataAccess.Emaitza;
15 import domain.Seller;
```

```
import exceptions.FileNotUploadedException;
import exceptions.MustBeLaterThanTodayException;
import exceptions.SaleAlreadyExistException;
import java.util.*;
import java.awt.image.BufferedImage;
import java.awt.Image;
import javax.imageio.ImageIO;
import java.io.IOException;
/** It implements the business logic as a web service. */
@WebService(endpointInterface = "businessLogic.RemoteBLFacade")
public class BLFacadeImplementation implements BLFacade {
    private static final String BASE_PATH="src/main/resources/images/";
    DataAccess dbManager;
    public BLFacadeImplementation() {
        System.out.println("Creating BLFacadeImplementation instance");
        dbManager=new DataAccess();
    }
    public BLFacadeImplementation(DataAccess da) {
        System.out.println("Creating BLFacadeImplementation instance with DataAccess parameter");
        dbManager=da;
    }
    @Override
    public patterns.iterator.ExtendedIterator<Sale> getSalesIterator(String desc) {
        return BLFacade.super.getSalesIterator(desc);
    }
    @WebMethod
    public Sale createSale(String title, String description,int status, float price, Date pubDate, String
sellerEmail, File file) throws FileNotUploadedException, MustBeLaterThanTodayException,
SaleAlreadyExistException {
        dbManager.open();

        Sale product=dbManager.createSale(title, description, status, price, pubDate, sellerEmail, file);
        dbManager.close();
        return product;
    }
    @WebMethod
    public List<Sale> getSales(String desc) {
        dbManager.open();
        List<Sale> rides=dbManager.getSales(desc);
        dbManager.close();
        return rides;
    }
    @WebMethod
    public List<Sale> getPublishedSales(String desc, Date pubDate) {
        dbManager.open();
        List<Sale> rides=dbManager.getPublishedSales(desc,pubDate);
        dbManager.close();
        return rides;
    }
    @WebMethod public BufferedImage getFile(String fileName) { return dbManager.getFile(fileName); }
    public void close() {
        DataAccess dB4oManager=new DataAccess();
        dB4oManager.close();
    }
    @WebMethod public void initializeBD() {
        dbManager.open();
        dbManager.initializeDB();
        dbManager.close();
    }
    @WebMethod public Image downloadImage(String imageName) {
        File image = new File(BASE_PATH+imageName);
        try { return ImageIO.read(image); }
        catch (IOException e) { throw new RuntimeException("Errorea gertatu da", e); }
    }
    @WebMethod public Emaitza isLogged(String log, String pass) {
        dbManager.open();
        Emaitza b = dbManager.isLogged(log, pass);
        dbManager.close();
        return b;
    }
    @WebMethod public Emaitza isAdminLogged(int ID, String pass) {
        dbManager.open();
        Emaitza b = dbManager.isAdminLogged(ID, pass);
        dbManager.close();
        return b;
    }

    @WebMethod public Emaitza isAdminRegistered(int ID, String pass) {
        dbManager.open();
        Emaitza b = dbManager.isAdminRegistered(ID, pass);
        dbManager.close();
        return b;
    }
    @WebMethod public Emaitza isRegistered(String mail, String user, String password) {
        dbManager.open();
        Emaitza b = dbManager.isRegistered(mail, user, password);
        dbManager.close();
        return b;
    }
    @WebMethod public String getEmail(String name, String password) {
        dbManager.open();
        return dbManager.getEmail(name, password);
    }
```

```
105     @WebMethod public void buySale(Sale s, Seller current) {
106         dbManager.open();
107         dbManager.buySale(s, current);
108     }
109     @WebMethod public boolean updateProfile(String email, String newName, String newPassword) {
110         dbManager.open();
111         boolean result = dbManager.updateProfile(email, newName, newPassword);
112         dbManager.close();
113         return result;
114     }
115     @WebMethod public Seller getSellerByEmail(String email) {
116         dbManager.open();
117         Seller s = dbManager.getSellerByEmail(email);
118         dbManager.close();
119         return s;
120     }
121     @WebMethod public void updateMoney(String email, float amount) {
122         dbManager.open();
123         dbManager.updateMoney(email, amount);
124         dbManager.close();
125     }
126     @WebMethod public List<Complaint> getUnmanagedComplaints() {
127         dbManager.open();
128         List<Complaint> complaints = dbManager.getUnmanagedComplaints();
129         dbManager.close();
130         return complaints;
131     }
132     @WebMethod public void markComplaintAsManaged(Integer complaintId) {
133         dbManager.open();

134         dbManager.markComplaintAsManaged(complaintId);
135         dbManager.close();
136     }
137     @WebMethod public void saveComplaint(String message, Seller complainant, Sale sale) {
138         dbManager.open();
139         dbManager.createComplaint(message, complainant, sale);
140         dbManager.close();
141     }
142     @WebMethod public void addProductToBasket(String email, int saleNumber) {
143         dbManager.open();
144         dbManager.addProductToBasket(email, saleNumber);
145         dbManager.close();
146     }
147     @WebMethod public void buyMultipleSales(List<Sale> sales, Seller buyer) {
148         dbManager.open();
149         try { for (Sale s : sales) dbManager.buySale(s, buyer); }
150         finally { dbManager.close(); }
151     }
152     @WebMethod public List<Sale> getBasket(String email) {
153         dbManager.open();
154         List<Sale> basket = dbManager.getBasket(email);
155         dbManager.close();
156         return basket;
157     }
158     @WebMethod public void clearBasket(String email) {
159         dbManager.open();
160         dbManager.clearBasket(email);
161         dbManager.close();
162     }
163     @WebMethod public void createEskaera(String title, String status, String description, String sellerMail)
 {
164         dbManager.open();
165         dbManager.createEskaera(title, status, description, sellerMail);
166         dbManager.close();
167     }
168     @WebMethod public List<Eskaera> getEskaerak(String desc) {
169         dbManager.open();
170         List<Eskaera> list1 = dbManager.getEskaerak(desc);
171         dbManager.close();
172         return list1;
173     }
174     @WebMethod public Eskaera getEskaeraByID(Integer id) {
175         dbManager.open();
176         Eskaera e = dbManager.getEskaeraByID(id);
177         dbManager.close();

178         return e;
179     }
180     @WebMethod public boolean createEskaeraOferta(Integer eskID, String selEmail, float amount, String
message) {
181         dbManager.open();
182         boolean a = dbManager.createEskaeraOferta(eskID, selEmail, amount, message);
183         dbManager.close();
184         return a;
185     }
186     @WebMethod public List<Eskaera> getMyEskaerak(Seller s) {
187         dbManager.open();
188         List<Eskaera> lista = dbManager.getMyEskaerak(s);
189         dbManager.close();
190         return lista;
191     }
192     @WebMethod public List<EskaeraOferta> getOffersByEskaera(Eskaera esk) {
193         dbManager.open();
```

```
        List<EskaeraOferta> lista = dbManager.getOffersByEskaera(esk);
        dbManager.close();
        return lista;
    }
    @WebMethod public Sale createSaleFromEskaeraOfer(Integer id) {
        dbManager.open();
        Sale s = dbManager.createSaleFromEskaeraOfer(id);
        dbManager.close();
        return s;
    }
    @WebMethod public void createReview(String authorEmail, int saleNumber, String comment, int stars) {
        dbManager.open();
        dbManager.createReview(authorEmail, saleNumber, comment, stars);
        dbManager.close();
    }
    @WebMethod public List<Sale> getBoughtsBySeller(String email) {
        dbManager.open();
        List<Sale> a =dbManager.getBoughtsBySeller(email);
        dbManager.close();
        return a;
    }
    @WebMethod public List<Sale> getSalesBySeller(String email) {
        dbManager.open();
        List<Sale> a =dbManager.getSalesBySeller(email);
        dbManager.close();
        return a;
    }
    @Override public List<Review> getReviewsBySeller(String email) {

        dbManager.open();
        List<Review> a=dbManager.getReviewsBySeller(email);
        dbManager.close();
        return a;
    }
    @WebMethod public String getSellerEmailBySale(Integer saleId) {
        dbManager.open();
        String s =dbManager.getSellerEmailBySale(saleId);
        dbManager.close();
        return s;
    }
}
```

src/main/java/businessLogic/BLFactory.java

Creator.create es el Factory Method; LocalCreator y RemoteCreator lo sobrescriben. La seleccion del modo no contiene la construccion concreta.

```
package businessLogic;
import java.util.Objects;
/** Selects a concrete Creator; presentation only supplies the desired mode. */
public final class BLFactory {
    public abstract static class Creator {
        public abstract BLFacade create();
    }
    public static final class LocalCreator extends Creator {
        @Override public BLFacade create() { return new BLFacadeImplementation(); }
    }
    public static final class RemoteCreator extends Creator {
        @Override public BLFacade create() { return RemoteFacade.connect(); }
    }
    private final Creator local;
    private final Creator remote;
    public BLFactory() { this(new LocalCreator(), new RemoteCreator()); }
    /** Tests may inject a Creator returning a Mockito mock without opening a database. */
    public BLFactory(Creator local, Creator remote) {
        this.local = Objects.requireNonNull(local, "local");
        this.remote = Objects.requireNonNull(remote, "remote");
    }
    public BLFacade getBusinessLogicFactory(boolean isLocal) {
        return (isLocal ? local : remote).create();
    }
}
```

src/main/java/businessLogic/RemoteBLFacade.java

Contrato SOAP original separado del estado local del iterador; conserva las 33 operaciones y el nombre XML BLFacade.

```
package businessLogic;
import java.io.File;
import java.util.Date;
import java.util.List;
import java.awt.Image;
import javax.jws.WebMethod;
import javax.jws.WebService;
import dataAccess.Emaitza;
import domain.*;
import exceptions.*;
```

```
 11 /** Existing SOAP contract, kept separate from the client-side iterator. */
 12 @WebService(name = "BLFacade", targetNamespace = "http://businessLogic/")
 13 public interface RemoteBLFacade {
 14     @WebMethod public Sale createSale(String title, String description, int status, float price, Date
pubDate, String sellerEmail, File file) throws FileNotUploadedException, MustBeLaterThanTodayException,
SaleAlreadyExistException;
 15     @WebMethod public List<Sale> getSales(String desc);
 16     @WebMethod public List<Sale> getPublishedSales(String desc, Date pubDate);
 17     @WebMethod public void initializeBD();
 18     @WebMethod public Image downloadImage(String imageName);
 19     @WebMethod public List<Complaint> getUnmanagedComplaints();
 20     @WebMethod public void markComplaintAsManaged(Integer complaintId);
 21     @WebMethod public Emaitza isLogged(String log, String password);
 22     @WebMethod public Emaitza isAdminLogged(int ID, String pass);
 23     @WebMethod public Emaitza isAdminRegistered(int ID, String pass);
 24     @WebMethod public Emaitza isRegistered(String mail, String user, String pass);
 25     @WebMethod public String getEmail(String name, String password);
 26     @WebMethod public void buySale(Sale s, Seller current);
 27     @WebMethod public boolean updateProfile(String email, String newName, String newPassword);
 28     @WebMethod public Seller getSellerByEmail(String email);
 29     @WebMethod public void updateMoney(String email, float amount);
 30     @WebMethod public void saveComplaint(String message, Seller complainant, Sale sale);
 31     @WebMethod public void addProductToBasket(String email, int saleNumber);
 32     @WebMethod public void buyMultipleSales(List<Sale> sales, Seller buyer);
 33     @WebMethod public List<Sale> getBasket(String email);
 34     @WebMethod public void clearBasket(String email);
 35     @WebMethod public void createEskaera(String title, String status, String description, String
sellerMail);
 36     @WebMethod public List<Eskaera> getEskaerak(String search);
 37     @WebMethod public Eskaera getEskaeraByID(Integer id);
 38     @WebMethod public boolean createEskaeraOferta(Integer eskID, String selEmail, float amount, String
message);
 39     @WebMethod public List<Eskaera> getMyEskaerak(Seller s);
 40     @WebMethod public List<EskaeraOferta> getOffersByEskaera(Eskaera esk);
 41     @WebMethod public Sale createSaleFromEskaeraOfer(Integer id);

 42     @WebMethod public void createReview(String authorEmail, int saleNumber, String comment, int stars);
 43     @WebMethod public List<Sale> getBoughtsBySeller(String email);
 44     @WebMethod public List<Sale> getSalesBySeller(String email);
 45     @WebMethod public List<Review> getReviewsBySeller(String email);
 46     @WebMethod public String getSellerEmailBySale(Integer saleId);
 47 }
```

src/main/java/businessLogic/RemoteFacade.java

Delegacion al puerto SOAP; el iterador heredado se crea en el cliente, no se serializa.

```
  1 package businessLogic;
  2 import java.io.File;
  3 import java.util.Date;
  4 import java.util.List;
  5 import java.awt.Image;
  6 import java.net.MalformedURLException;
  7 import java.net.URL;
  8 import java.util.Objects;
  9 import javax.xml.namespace.QName;
 10 import javax.xml.ws.Service;
 11 import configuration.ConfigXML;
 12 import dataAccess.Emaitza;
 13 import domain.*;
 14 import exceptions.*;
 15 /** ConcreteProduct: SOAP delegate plus the inherited client-side iterator. */
 16 public final class RemoteFacade implements BLFacade {
 17     private final RemoteBLFacade service;
 18     public RemoteFacade(RemoteBLFacade service) {
 19         this.service = Objects.requireNonNull(service, "service");
 20     }
 21     public static BLFacade connect() {
 22         ConfigXML c = ConfigXML.getInstance();
 23         String wsdl = "http://" + c.getBusinessLogicNode() + ":" + c.getBusinessLogicPort()
 24             + "/ws/" + c.getBusinessLogicName() + "?wsdl";
 25         try {
 26             Service s = Service.create(new URL(wsdl),
 27                 new QName("http://businessLogic/", "BLFacadeImplementationService"));
 28             return new RemoteFacade(s.getPort(RemoteBLFacade.class));
 29         } catch (MalformedURLException e) {
 30             throw new IllegalArgumentException("Invalid business logic URL: " + wsdl, e);
 31         }
 32     }
 33     @Override public Sale createSale(String title, String description, int status, float price, Date
pubDate, String sellerEmail, File file) throws FileNotUploadedException, MustBeLaterThanTodayException,
SaleAlreadyExistException { return service.createSale(title, description, status, price, pubDate, sellerEmail,
file); }
 34     @Override public List<Sale> getSales(String desc) { return service.getSales(desc); }
 35     @Override public List<Sale> getPublishedSales(String desc, Date pubDate) { return
service.getPublishedSales(desc, pubDate); }
 36     @Override public void initializeBD() { service.initializeBD(); }
 37     @Override public Image downloadImage(String imageName) { return service.downloadImage(imageName); }
 38     @Override public List<Complaint> getUnmanagedComplaints() { return service.getUnmanagedComplaints(); }
 39     @Override public void markComplaintAsManaged(Integer complaintId) {
service.markComplaintAsManaged(complaintId); }
```

```
 40     @Override public Emaitza isLogged(String log, String password) { return service.isLogged(log, password);

 }
 41     @Override public Emaitza isAdminLogged(int ID, String pass) { return service.isAdminLogged(ID, pass); }
 42     @Override public Emaitza isAdminRegistered(int ID, String pass) { return service.isAdminRegistered(ID,
pass); }
 43     @Override public Emaitza isRegistered(String mail, String user, String pass) { return
service.isRegistered(mail, user, pass); }
 44     @Override public String getEmail(String name, String password) { return service.getEmail(name,
password); }
 45     @Override public void buySale(Sale s, Seller current) { service.buySale(s, current); }
 46     @Override public boolean updateProfile(String email, String newName, String newPassword) { return
service.updateProfile(email, newName, newPassword); }
 47     @Override public Seller getSellerByEmail(String email) { return service.getSellerByEmail(email); }
 48     @Override public void updateMoney(String email, float amount) { service.updateMoney(email, amount); }
 49     @Override public void saveComplaint(String message, Seller complainant, Sale sale) {
service.saveComplaint(message, complainant, sale); }
 50     @Override public void addProductToBasket(String email, int saleNumber) {
service.addProductToBasket(email, saleNumber); }
 51     @Override public void buyMultipleSales(List<Sale> sales, Seller buyer) { service.buyMultipleSales(sales,
 buyer); }
 52     @Override public List<Sale> getBasket(String email) { return service.getBasket(email); }
 53     @Override public void clearBasket(String email) { service.clearBasket(email); }
 54     @Override public void createEskaera(String title, String status, String description, String sellerMail)
{ service.createEskaera(title, status, description, sellerMail); }
 55     @Override public List<Eskaera> getEskaerak(String search) { return service.getEskaerak(search); }
 56     @Override public Eskaera getEskaeraByID(Integer id) { return service.getEskaeraByID(id); }
 57     @Override public boolean createEskaeraOferta(Integer eskID, String selEmail, float amount, String
message) { return service.createEskaeraOferta(eskID, selEmail, amount, message); }
 58     @Override public List<Eskaera> getMyEskaerak(Seller s) { return service.getMyEskaerak(s); }
 59     @Override public List<EskaeraOferta> getOffersByEskaera(Eskaera esk) { return
service.getOffersByEskaera(esk); }
 60     @Override public Sale createSaleFromEskaeraOfer(Integer id) { return
service.createSaleFromEskaeraOfer(id); }
 61     @Override public void createReview(String authorEmail, int saleNumber, String comment, int stars) {
service.createReview(authorEmail, saleNumber, comment, stars); }
 62     @Override public List<Sale> getBoughtsBySeller(String email) { return service.getBoughtsBySeller(email);
 }
 63     @Override public List<Sale> getSalesBySeller(String email) { return service.getSalesBySeller(email); }
 64     @Override public List<Review> getReviewsBySeller(String email) { return
service.getReviewsBySeller(email); }
 65     @Override public String getSellerEmailBySale(Integer saleId) { return
service.getSellerEmailBySale(saleId); }
 66 }
```

src/main/java/gui/ApplicationLauncher.java

La presentacion decide el modo con ConfigXML y obtiene la fachada mediante BLFactory. La ventana se crea en el EDT.

```
 1 package gui;
 2 import java.awt.Color;
 3 import java.util.Locale;
 4 import javax.swing.SwingUtilities;
 5 import javax.swing.UIManager;
 6 import businessLogic.BLFactory;
 7 import businessLogic.BLFacade;
 8 import configuration.ConfigXML;
 9 public class ApplicationLauncher {
10     public static void main(String[] args) {
11         ConfigXML c = ConfigXML.getInstance();
12         Locale.setDefault(new Locale(c.getLocale()));
13         BLFacade facade;
14         try {
15             // Potential DB/network work stays off the Swing event-dispatch thread.
16             facade = new BLFactory().getBusinessLogicFactory(c.isBusinessLogicLocal());
17         } catch (Exception e) {
18             SwingUtilities.invokeLater(() -> {
19                 MainGUI window = new MainGUI("");
20                 window.jLabelSelectOption.setText("Error: " + e.toString());
21                 window.jLabelSelectOption.setForeground(Color.RED);
22                 window.setVisible(true);
23             });
24             return;
25         }
26         SwingUtilities.invokeLater(() -> {
27             try { UIManager.setLookAndFeel("javax.swing.plaf.metal.MetalLookAndFeel"); }
28             catch (Exception e) { System.err.println(e); }
29             MainGUI.setBussinessLogic(facade);
30             new MainGUI("").setVisible(true);
31         });
32     }
33 }
```

src/main/java/gui/SellerAdapter.java

getValueAt traduce los campos de Sale a celdas. getColumnClass declara tipos sin depender de que exista una primera fila.

```
 1 package gui;
 2 import java.util.ArrayList;
 3 import java.util.Date;
 4 import java.util.List;
 5 import java.util.Objects;
 6 import javax.swing.table.AbstractTableModel;
 7 import domain.Sale;
 8 import domain.Seller;
 9 /** Object Adapter: Seller/Sale -> Swing TableModel. */
10 public final class SellerAdapter extends AbstractTableModel {
11     private static final long serialVersionUID = 1L;
12     private static final String[] COLUMNS = {"Sale", "Title", "Description", "Status", "Price", "Publication
date"};
13     private static final Class<?>[] TYPES = {Integer.class, String.class, String.class, Integer.class,
Float.class, Date.class};
14     private final List<Sale> sales;
15     public SellerAdapter(Seller seller) {
16         this(Objects.requireNonNull(seller, "seller"), seller.getSales());
17     }
18     /** Remote sellers omit sales in JAXB; pass getSalesBySeller(email) explicitly. */
19     public SellerAdapter(Seller seller, List<Sale> sales) {
20         Objects.requireNonNull(seller, "seller");
21         this.sales = new ArrayList<Sale>(Objects.requireNonNull(sales, "sales"));
22     }
23     @Override public int getRowCount() { return sales.size(); }
24     @Override public int getColumnCount() { return COLUMNS.length; }
25     @Override public String getColumnName(int column) { return COLUMNS[column]; }
26     @Override public Class<?> getColumnClass(int column) { return TYPES[column]; }
27     @Override public boolean isCellEditable(int row, int column) { return false; }
28     @Override public Object getValueAt(int row, int column) {
29         Sale s = sales.get(row);
30         switch (column) {
31         case 0: return s.getSaleNumber();
32         case 1: return s.getTitle();
33         case 2: return s.getDescription();
34         case 3: return s.getStatus();
35         case 4: return s.getPrice();
36         case 5: return s.getPublicationDate();
37         default: throw new IndexOutOfBoundsException("column: " + column);
38         }
39     }
40 }
```

src/main/java/gui/SellerTable.java

JTable recibe SellerAdapter como TableModel. La sobrecarga permite pasar ventas obtenidas del servidor remoto.

```
 1 package gui;
 2 import java.awt.BorderLayout;
 3 import java.awt.Dimension;
 4 import java.util.List;
 5 import javax.swing.JFrame;
 6 import javax.swing.JScrollPane;
 7 import javax.swing.JTable;
 8 import domain.Sale;
 9 import domain.Seller;
10 public final class SellerTable extends JFrame {
11     private static final long serialVersionUID = 1L;
12     public SellerTable(Seller seller) { this(seller, seller.getSales()); }
13     public SellerTable(Seller seller, List<Sale> sales) {
14         super(seller.getName() + "'s sales");
15         JTable table = new JTable(new SellerAdapter(seller, sales));
16         table.setAutoCreateRowSorter(true);
17         table.setPreferredScrollableViewportSize(new Dimension(850, 180));
18         add(new JScrollPane(table), BorderLayout.CENTER);
19         setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
20         pack();
21         setLocationByPlatform(true);
22     }
23 }
```

src/main/java/patterns/demo/PatternsDemo.java

Demostracion inversa y directa; resuelve Seller por email en lugar de suponer que isLogged devuelve Seller.

```
 1 package patterns.demo;
 2 import businessLogic.BLFacade;
 3 import businessLogic.BLFactory;
 4 import configuration.ConfigXML;
 5 import domain.Sale;
 6 import domain.Seller;
 7 import gui.SellerTable;
 8 import java.util.List;
 9 import javax.swing.SwingUtilities;
10 import patterns.iterator.ExtendedIterator;
```

```
/** Real application demo: optional seller email and search text, no hard-coded credentials. */
public final class PatternsDemo {
    public static String describe(ExtendedIterator<Sale> it) {
        StringBuilder out = new StringBuilder("FROM LAST TO FIRST\n");
        it.goLast();
        while (it.hasPrevious()) out.append(it.previous()).append('\n');
        out.append("\nFROM FIRST TO LAST\n");
        it.goFirst();
        while (it.hasNext()) out.append(it.next()).append('\n');
        return out.toString();
    }
    public static void main(String[] args) {
        BLFacade bl = new
BLFactory().getBusinessLogicFactory(ConfigXML.getInstance().isBusinessLogicLocal());
        System.out.print(describe(bl.getSalesIterator(args.length > 1 ? args[1] : "")));
        if (args.length > 0) {
            Seller seller = bl.getSellerByEmail(args[0]);
            if (seller == null) throw new IllegalArgumentException("Seller not found: " + args[0]);
            List<Sale> sales = bl.getSalesBySeller(seller.getEmail());
            SwingUtilities.invokeLater(() -> new SellerTable(seller, sales).setVisible(true));
        }
    }
}
```

src/main/java/patterns/iterator/ExtendedIterator.java

Contrato generico solicitado, ampliando Iterator con recorrido inverso y reposicionamiento.

```
package patterns.iterator;
import java.util.Iterator;
/** Bidirectional traversal. goFirst/goLast reset the next element to return. */
public interface ExtendedIterator<T> extends Iterator<T> {
    T previous();
    boolean hasPrevious();
    void goFirst();
    void goLast();
}
```

src/main/java/patterns/iterator/ListExtendedIterator.java

cursor es el elemento actual; previous devuelve antes de decrementar y goLast usa size-1. Se comprueban limites y remove no esta permitido.

```
package patterns.iterator;
import java.util.ArrayList;
import java.util.List;
import java.util.NoSuchElementException;
import java.util.Objects;
/** Snapshot of list membership; cursor denotes the next element in either direction. */
public final class ListExtendedIterator<T> implements ExtendedIterator<T> {
    private final List<T> elements;
    private int cursor;
    public ListExtendedIterator(List<? extends T> elements) {
        this.elements = new ArrayList<T>(Objects.requireNonNull(elements, "elements"));
        goFirst();
    }
    @Override public boolean hasNext() { return cursor >= 0 && cursor < elements.size(); }
    @Override public boolean hasPrevious() { return cursor >= 0 && cursor < elements.size(); }
    @Override public T next() {
        if (!hasNext()) throw new NoSuchElementException();
        return elements.get(cursor++);
    }
    @Override public T previous() {
        if (!hasPrevious()) throw new NoSuchElementException();
        return elements.get(cursor--);
    }
    @Override public void goFirst() { cursor = 0; }
    @Override public void goLast() { cursor = elements.size() - 1; }
    @Override public void remove() { throw new UnsupportedOperationException("Read-only snapshot"); }
}
```

src/test/java/patterns/PatternsScreenshotTest.java

Ejecuta y captura ventanas Swing reales; la omision por entorno headless impide certificar la entrega.

```
package patterns;
import static org.junit.Assert.*;
import java.awt.*;
import java.awt.image.BufferedImage;
import java.io.File;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.util.Date;
import javax.imageio.ImageIO;
import javax.swing.*;
import org.junit.Assume;
import org.junit.Test;
```

```
import domain.*;
import gui.SellerTable;
import patterns.demo.PatternsDemo;
import patterns.iterator.ListExtendedIterator;
/** Screenshots of real Swing windows rendered by Java, not mock images. */
public class PatternsScreenshotTest {
    private void capture(JFrame frame, String name) throws Exception {
        try {
            SwingUtilities.invokeAndWait(() -> frame.setVisible(true));
            Robot robot = new Robot(); robot.waitForIdle(); robot.delay(600);
            Rectangle[] bounds = new Rectangle[1];
            SwingUtilities.invokeAndWait(() -> bounds[0] = frame.getBounds());
            BufferedImage image = robot.createScreenCapture(bounds[0]);
            assertTrue(ImageIO.write(image, "png", new File("doc/patterns/" + name + ".png")));
        } finally { SwingUtilities.invokeAndWait(() -> frame.dispose()); }
    }
    @Test public void captureExecutedExamples() throws Exception {
        Assume.assumeFalse("Run with xvfb-run to capture real windows", GraphicsEnvironment.isHeadless());
        Files.createDirectories(new File("doc/patterns").toPath());
        Seller seller = new Seller("seller1@example.test", "Demo Seller", "unused");
        Sale a = seller.addSale("Bicycle", "City bicycle", 1, 120f, new Date(0), null);
        Sale b = seller.addSale("Monitor", "24 inch", 0, 85f, new Date(86400000L), null);
        Sale c = seller.addSale("Keyboard", "Mechanical", 1, 35f, new Date(172800000L), null);
        a.setSaleNumber(1); b.setSaleNumber(2); c.setSaleNumber(3);
        String output = PatternsDemo.describe(new ListExtendedIterator<Sale>(seller.getSales()));
        assertTrue(output.indexOf("3;Keyboard") < output.indexOf("1;Bicycle"));
        Files.write(new File("doc/patterns/iterator-output.txt").toPath(),
output.getBytes(StandardCharsets.UTF_8));
        JFrame[] windows = new JFrame[2];
        SwingUtilities.invokeAndWait(() -> {
            windows[0] = new SellerTable(seller);
            windows[0].setLocation(30,30);
            JFrame console = new JFrame("Iterator - executed example (deterministic fixture)");

            JTextArea text = new JTextArea(output);
            text.setEditable(false); text.setFont(new Font(Font.MONOSPACED, Font.PLAIN, 17));
            console.add(new JScrollPane(text)); console.setSize(730,360); console.setLocation(30,300);
            windows[1] = console;
        });
        capture(windows[0], "adapter");
        capture(windows[1], "iterator");
    }
}
```

src/test/java/patterns/PatternsTest.java

Ocho pruebas unitarias de Factory, Iterator y Adapter con mocks para no tocar bases de datos.

```
package patterns;
import static org.junit.Assert.*;
import static org.mockito.Mockito.*;
import java.util.*;
import org.junit.Test;
import businessLogic.*;
import dataAccess.DataAccess;
import domain.*;
import gui.SellerAdapter;
import patterns.iterator.*;
public class PatternsTest {
    @Test public void reverseThenForwardAndBounds() {
        ExtendedIterator<Integer> it = new ListExtendedIterator<Integer>(Arrays.asList(1,2,3));
        it.goLast();
        List<Integer> backwards = new ArrayList<Integer>();
        while(it.hasPrevious()) backwards.add(it.previous());
        assertEquals(Arrays.asList(3,2,1), backwards);
        assertThrows(NoSuchElementException.class, () -> it.previous());
        it.goFirst();
        List<Integer> forwards = new ArrayList<Integer>();
        while(it.hasNext()) forwards.add(it.next());
        assertEquals(Arrays.asList(1,2,3), forwards);
        assertThrows(NoSuchElementException.class, () -> it.next());
        assertThrows(UnsupportedOperationException.class, () -> it.remove());
    }
    @Test public void emptyAndSingleton() {
        ExtendedIterator<Integer> empty = new ListExtendedIterator<Integer>(Collections.emptyList());
        assertFalse(empty.hasNext());
        assertFalse(empty.hasPrevious());
        empty.goLast();
        assertFalse(empty.hasPrevious());
        assertThrows(NoSuchElementException.class, () -> empty.previous());
        ExtendedIterator<Integer> one = new ListExtendedIterator<Integer>(Arrays.asList(7));
        assertEquals(Integer.valueOf(7), one.next());
        assertFalse(one.hasNext());
        one.goLast();
        assertEquals(Integer.valueOf(7), one.previous());
        assertFalse(one.hasPrevious());
    }
    @Test public void snapshotAndDirectionChange() {
        List<Integer> input = new ArrayList<Integer>(Arrays.asList(1,2,3));
        ExtendedIterator<Integer> it = new ListExtendedIterator<Integer>(input);
        input.clear();
```

```
        assertEquals(Integer.valueOf(1), it.next());
        assertEquals(Integer.valueOf(2), it.previous());

        assertEquals(Integer.valueOf(1), it.previous());
        it.goLast();
        assertEquals(Integer.valueOf(3), it.next());
    }
    @Test public void factoryUsesInjectedCreatorsWithoutDatabase() {
        BLFacade local = mock(BLFacade.class), remote = mock(BLFacade.class);
        BLFactory.Creator lc = mock(BLFactory.Creator.class), rc = mock(BLFactory.Creator.class);
        when(lc.create()).thenReturn(local); when(rc.create()).thenReturn(remote);
        BLFactory factory = new BLFactory(lc,rc);
        assertSame(local, factory.getBusinessLogicFactory(true));
        verify(rc, never()).create();
        assertSame(remote, factory.getBusinessLogicFactory(false));
        verify(lc).create(); verify(rc).create();
    }
    @Test public void localFacadeUsesExistingSearch() {
        DataAccess db = mock(DataAccess.class);
        Sale sale = new Sale();
        when(db.getSales("bike")).thenReturn(Arrays.asList(sale));
        BLFacade bl = new BLFacadeImplementation(db);
        assertSame(sale, bl.getSalesIterator("bike").next());
        org.mockito.InOrder order = inOrder(db);
        order.verify(db).open(); order.verify(db).getSales("bike"); order.verify(db).close();
    }
    @Test public void remoteIteratorStaysOnClient() {
        RemoteBLFacade soap = mock(RemoteBLFacade.class);
        Sale sale = new Sale();
        when(soap.getSales("bike")).thenReturn(Arrays.asList(sale));
        when(soap.getSales("")).thenReturn(Arrays.asList(sale));
        BLFacade bl = new RemoteFacade(soap);
        assertSame(sale, bl.getSalesIterator("bike").next());
        assertSame(sale, bl.getSalesIterator().next());
        verify(soap).getSales("bike"); verify(soap).getSales("");
        verifyNoMoreInteractions(soap);
    }
    @Test public void sellerAdapterMapsValuesAndTypes() {
        Seller seller = new Seller("demo@example.test", "Demo", "unused");
        Date date = new Date(0);
        Sale sale = seller.addSale("Bike", "Demo", 1, 42.5f, date, null);
        sale.setSaleNumber(9);
        SellerAdapter adapter = new SellerAdapter(seller);
        assertEquals(1, adapter.getRowCount()); assertEquals(6, adapter.getColumnCount());
        Object[] expected = {9, "Bike", "Demo", 1, 42.5f, date};
        for(int column=0; column<expected.length; column++) {
            assertEquals(expected[column], adapter.getValueAt(0,column));
            assertTrue(adapter.getColumnClass(column).isInstance(expected[column]));

            assertFalse(adapter.isCellEditable(0,column));
        }
        seller.addSell(new Sale());
        assertEquals(1, adapter.getRowCount());
        assertThrows(IndexOutOfBoundsException.class, () -> adapter.getValueAt(0,6));
    }
    @Test public void emptyAndRemoteSellerAdapter() {
        Seller seller = new Seller("demo@example.test", "Demo", "unused");
        SellerAdapter empty = new SellerAdapter(seller);
        assertEquals(0, empty.getRowCount()); assertEquals(Float.class, empty.getColumnClass(4));
        SellerAdapter remote = new SellerAdapter(seller, Arrays.asList(new Sale()));
        assertEquals(1, remote.getRowCount());
        assertNull(remote.getValueAt(0,0));
    }
}
```

Limitaciones y entrega

No se certifican SOAP extremo a extremo ni ObjectDB real. Las cabeceras graficas del enunciado requieren contraste visual. El Word es adicional al patters.pdf exigido. No se ha subido ningun archivo a OneDrive.